

Part 4

Vector Database

Basics



01

Overview

- 벡터 데이터베이스의 정의
- 벡터 DB 파이프라인
- 유의 사항

02

Indexing

- Indexing의 정의
- Indexing 알고리즘 종류

03

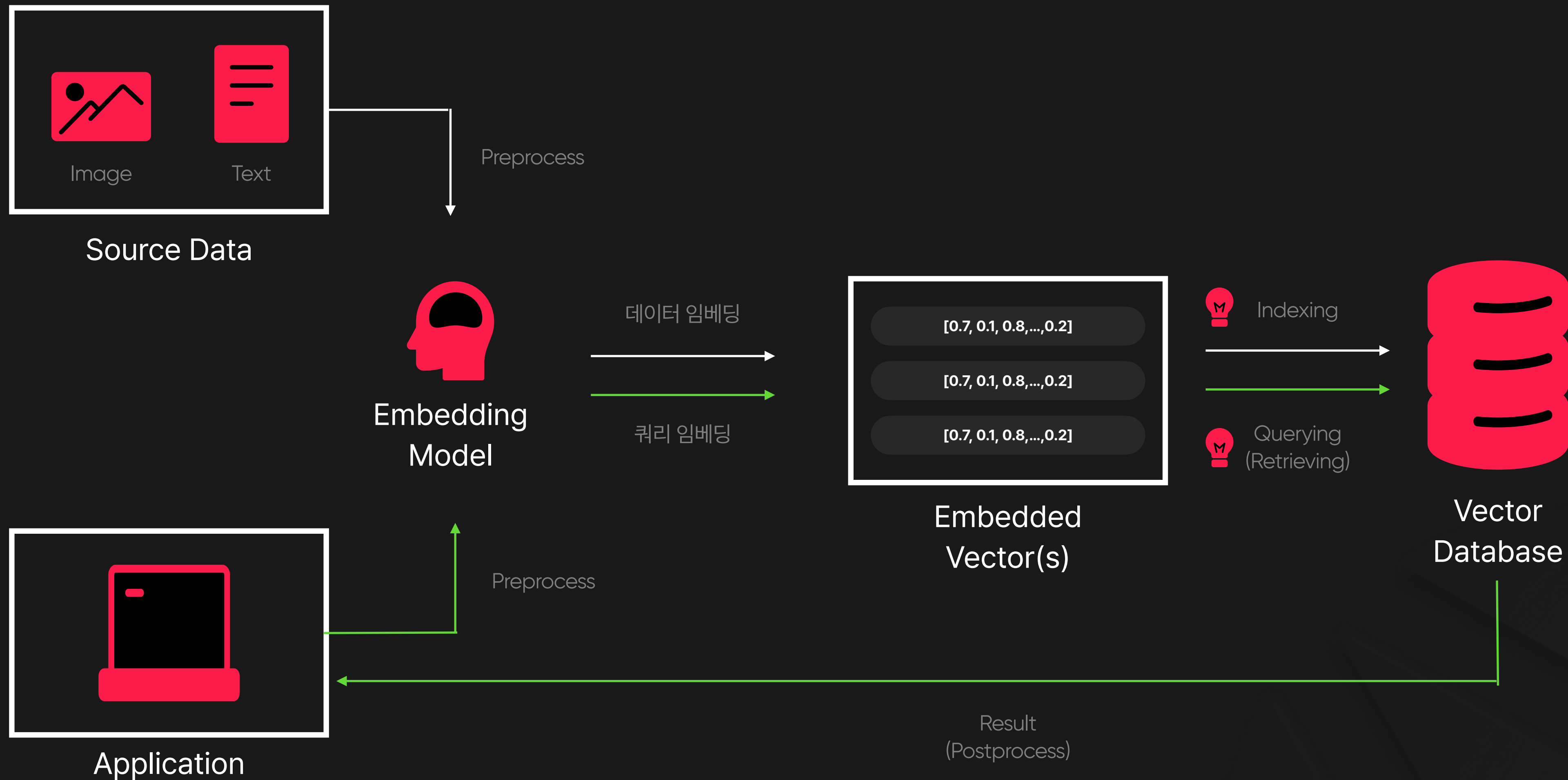
Querying

- Querying의 정의
- Querying 종류
- 실습

Overview

Vector Database란?

- 텍스트, 이미지, 오디오 등 임베딩 방식으로 나타낸 데이터를 저장하고, 관리하고, 검색하는 기능을 제공하는 임베딩 벡터 전용 DB.
- 고차원(벡터 차원 수)의 공간(인덱스)에 임베디드 벡터를 인덱싱하여 저장하는 방식.
- 입력 Query와 가장 가까운 이웃을 찾아주는 검색 방식. ANN(Approximate Nearest Neighbor) 검색 알고리즘에 기반한 검색 효율성 도모.
- CRUD(Create, Read, Update, Delete) 지원
- 벡터DB 서비스 제공자 마다 약간씩 다른 인덱싱, 검색 방식을 채택하여 제공.
- 모든 Vector Database는 공통적인 단계들로 (1) Indexing (2) Querying 를 따르며, 앞 뒤 사이사이에 Loading, Transforming, Post-Processing 등 검색 성능을 높이기 위한 처리작업들을 VectorDB 자체적으로 지원하는 것도 있고 따로 해줘야 하는 것들도 있음.



Vector Database Pipeline

관계형 DB

벡터 DB

행
(레코드)



벡터

[0.7, 0.1, 0.8,...,0.2]

열
(컬럼)



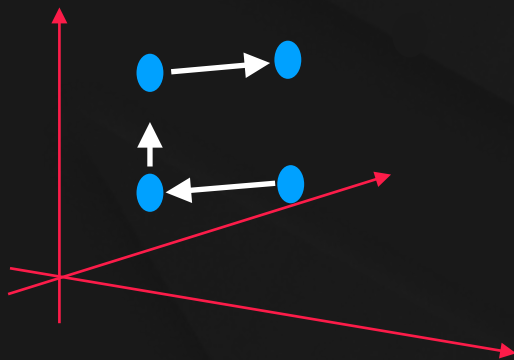
차원

[0.7, 0.1, 0.8,...,0.2]

테이블



인덱스
(컬렉션)



관계형 DB → 벡터 DB

Vector Index

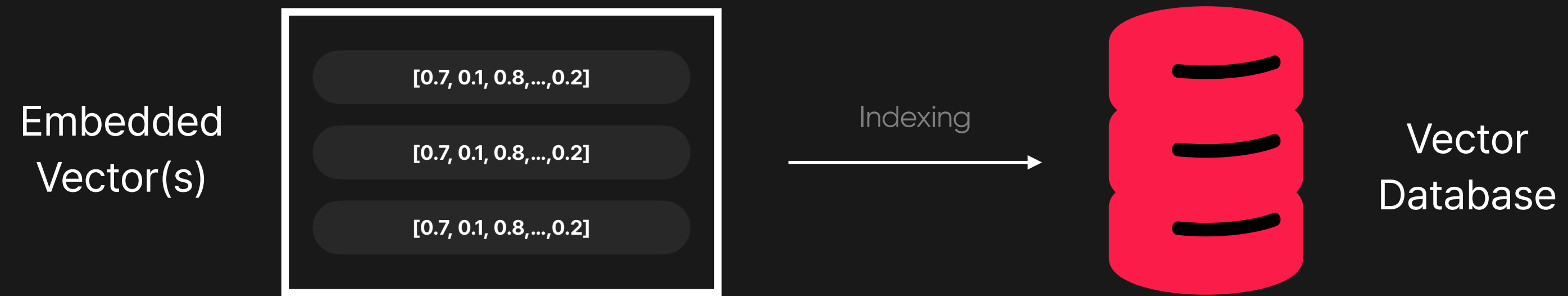
- ANN 기반의 Vector 인덱싱, 쿼링 기능 제공
- 스토리지 미제공으로 파일시스템에 자체적인 오브젝트(pkI등) 형태로 저장, 메모리로 저장된 인덱스 로딩 필요
- 메타데이터 기반의 하이브리드 서치 기능 미제공(혹은 제한적 제공)
- 데이터 매니징 측면에서의 한계로 빠른 프로토타이핑 용도로만 사용 추천
- FAISS, Annoy 등

Vector DB

- ANN 기반의 Vector 인덱싱, 쿼링 기능 제공
- 스토리지 함께 제공 및 데이터 관리 기능 제공(Insert, Delete, Update)
- 메타데이터 기반의 하이브리드 서치 기능 제공
- 데이터 보안, 액세스 관리 등 용이
- 프로덕션 레벨 어플리케이션 구성 시 사용

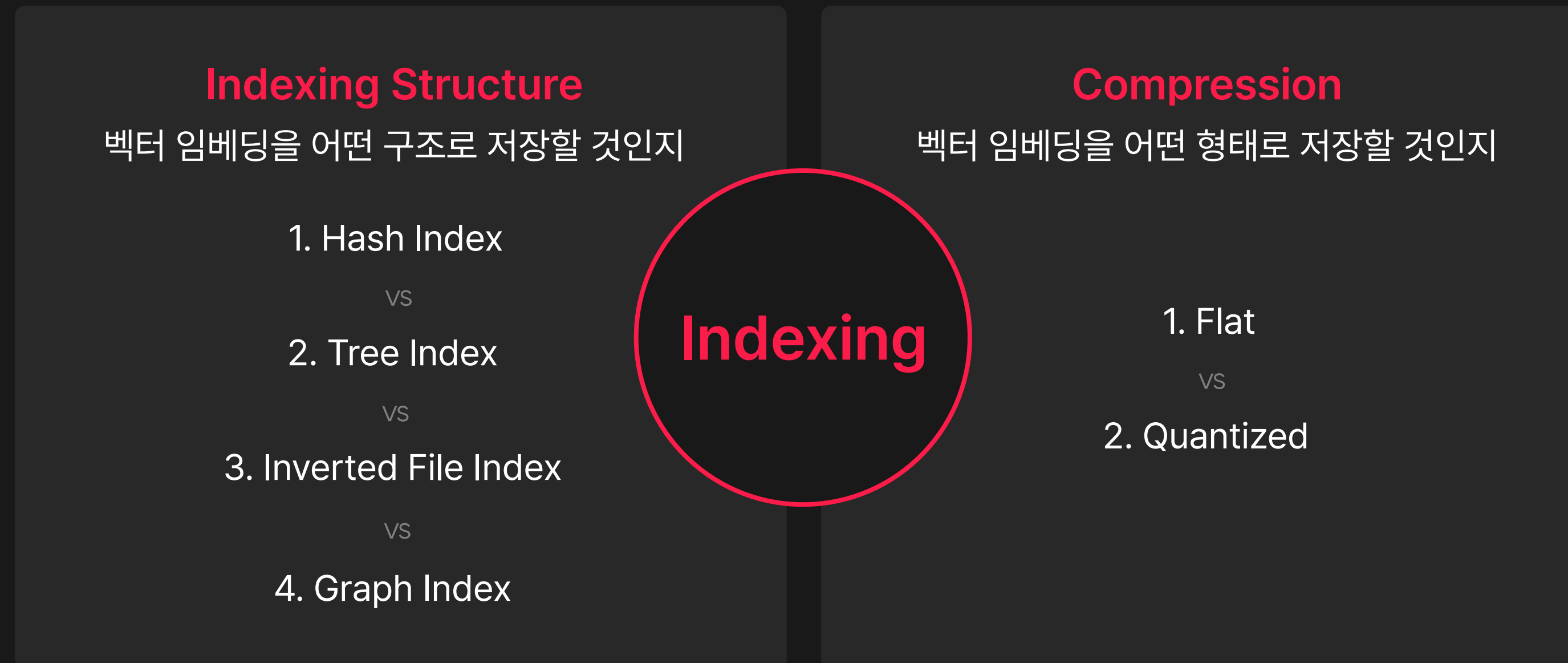
Vector Index vs Vector Database

Indexing



Indexing 이란?

- 데이터베이스에 벡터 데이터를 구조화된 인덱스에 담는 행위
- 추후 검색 성능을 고려하여 KNN이 아닌, ANN(Approximate Nearest Neighbor) 가능한 구조로 보통 설계
- GOAL : 검색 정확도 ↔ 검색 속도 tradeoff 관계 최적화



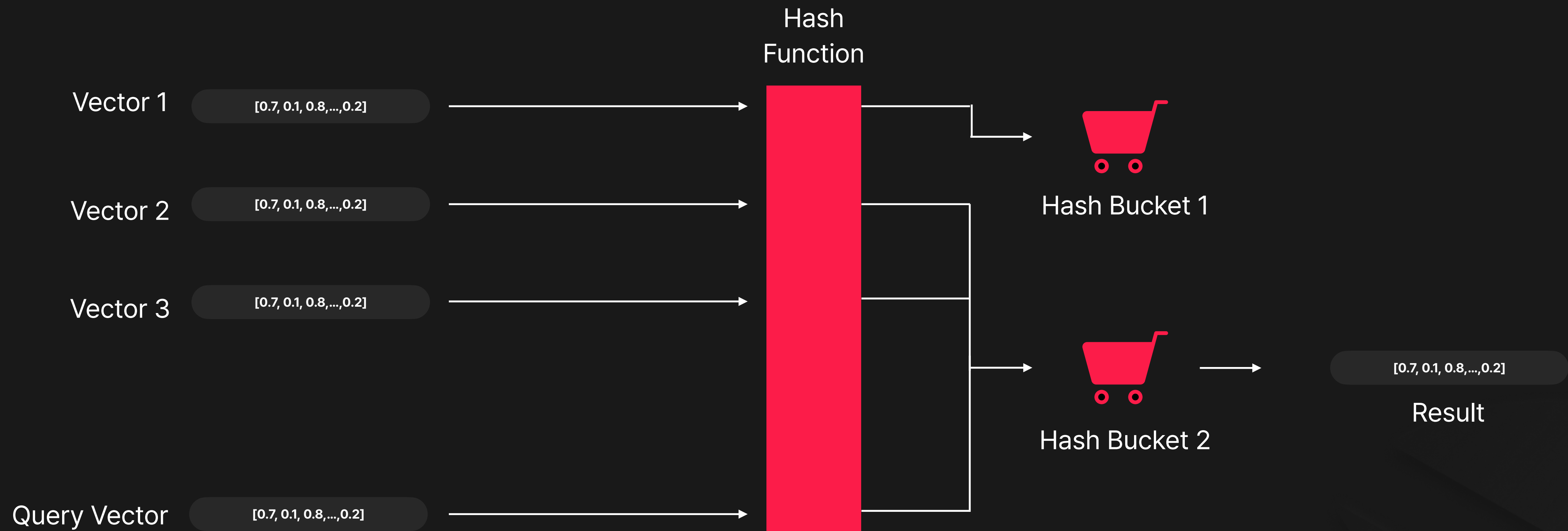
Indexing Components

Hash Index

- 고차원 데이터를 저차원 해시코드로 변환하여 서칭컴플렉시티 개선을 노리는 방식
- 일반적 해싱과 반대로, 벡터 인덱싱 시 비슷한 데이터끼리 해시충돌이 나도록 하는 구조
- 이때 사용되는 해싱함수를 그대로 쿼리 벡터에 대해서도 적용시켜 동일한 해시버킷에 위치한 벡터들에 대해서만 거리계산 수행
- LSH(Locally Sensitive Hashing) 등이 여기에 해당

장점: 검색 속도가 매우 빠르다

단점: 검색 정확도가 좋지 않다



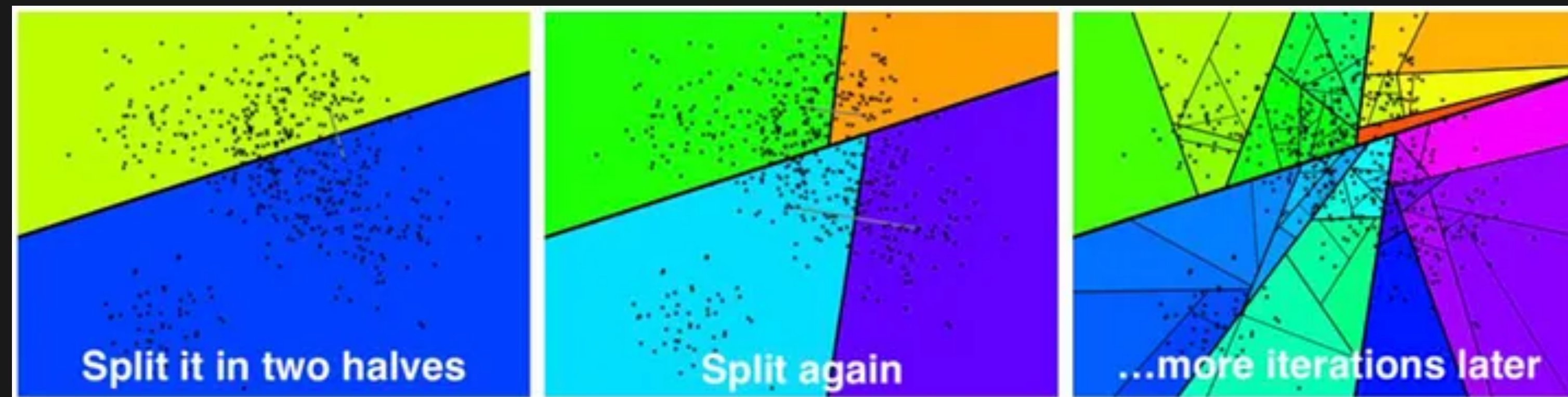
Hash Index

Tree Index

- Binary Search Tree 구조를 사용하여 고차원 벡터 공간에서의 검색 속도 향상을 도모하는 방식
- 유사한 벡터들이 같은 서브트리 노드(혹은 공간)에 속하도록 하는 구조
- 검색 시 해시버킷과 유사하게 쿼링 벡터가 속하는 서브트리 노드에 존재하는 다른 벡터들간의 거리만 계산하여 검색 속도 최적화가 이루어짐
- Spotify Annoy 알고리즘이 여기에 해당

장점: 검색 속도가 매우 빠르다

단점: 고차원 벡터에 대해서는 검색 정확도가 좋지 않다



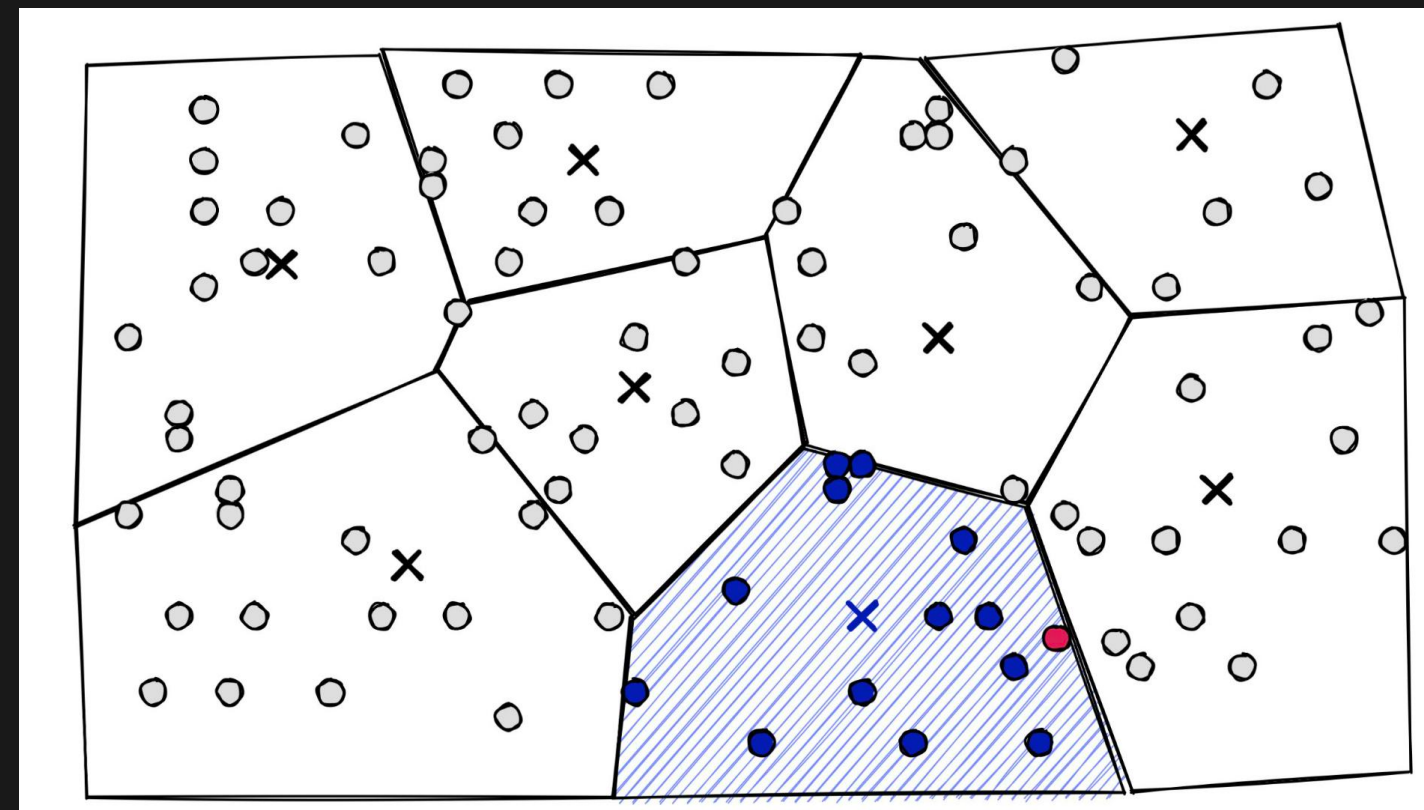
Annoy

Inverted File(IVF) Index

- 벡터 공간을 보로노이 다이어그램이라고 하는 n개의 공간으로 나누어서 서칭스페이스를 줄이는 효과를 도모하는 방식
- 군집화 하는 효과와 유사
- 쿼리 벡터에 대해 유사 벡터 검색 시, 벡터 to 벡터 대신 보로노이 다이어그램별 Centroid 포인트들의 좌표와 거리 계산을 하여 가장 가까운 centroid가 해당하는 공간 내 임베딩 벡터들과 거리계산을 하는 구조

장점: 검색 속도가 매우 빠름

단점: 공간을 형성하는 벡터가 많을수록 공간 나누는 작업의 속도가 느려짐



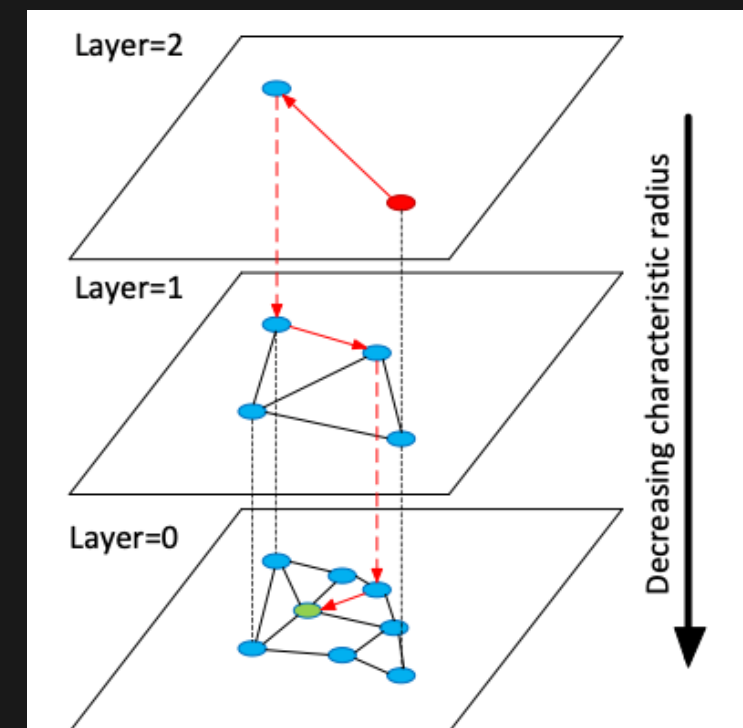
IVF

Graph Index

- 임베딩 벡터들을 그래프 구조로 구조화하여 임베딩 벡터를 노드로, 연결되는 엣지를 벡터간(노드간) 거리로 표현
- 유사한 벡터끼리 엣지 커넥션이 더 잘 이루어 지도록 설계
- HNSW(Hierarchical Navigable Small World)와 같은 알고리즘들이 여기에 해당
 - : 여러 계층으로 그래프 제작
 - : 가장 밀도가 낮은 계층에서 랜덤한 노드로 시작하여, 해당 계층에서 가장 가까운 노드를 찾은 후 다음 계층으로 이동
 - : 최종적으로 원래 쿼리 벡터노드와 가장 가까운 이웃을 찾을 때 까지 계층 탐색

장점: 검색 속도도 빠르고 검색 성능도 빠르다

단점: 검색 그래프를 구성하는 방식에 따라 검색 성능 의존적 (파라미터 튜닝 필요)



HNSW

Flat

- 임베딩 벡터를 원 형상 그대로의 형태로 두는 방식.
- 원본 그대로를 사용하기 때문에 Brute-force한 거리계산이 벡터간 수행됨
- Approximate이 아닌, kNN Full Search가 들어가기 때문에 데이터의 크기에 따라 계산 Complexity도 선형적으로 증가

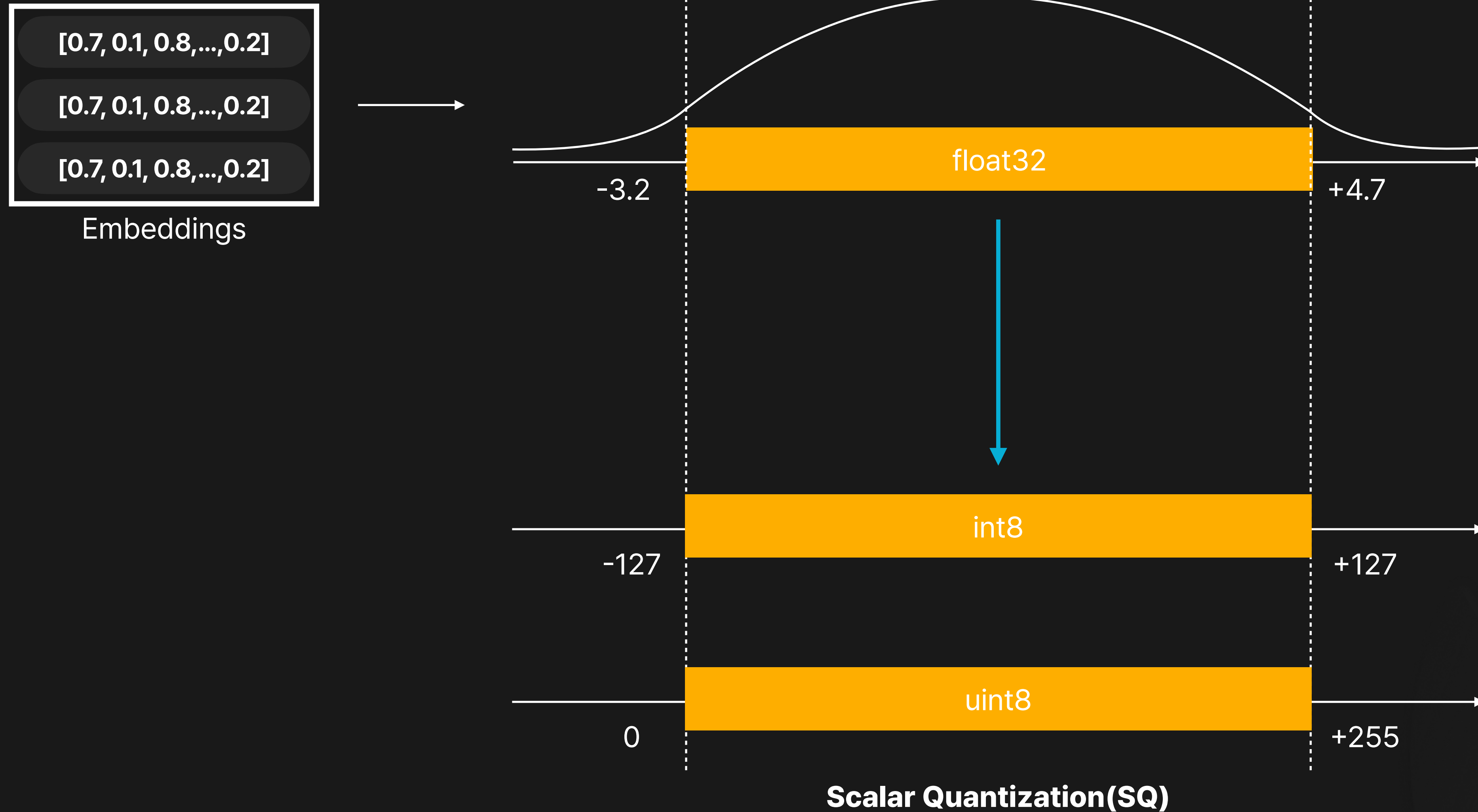
Quantized

- 임베딩 벡터에 대해 더 작은 단위의 바이트로 구성된 청크들로 변환시키는 작업
- 양자화라고 부르며, 보통 float을 int로 변환
- 메모리 효율화를 통한 검색성능 개선
- 크게 두가지 유형으로 사용됨: Scalar Quantization(SQ), Product Quantization(PQ)

Compression

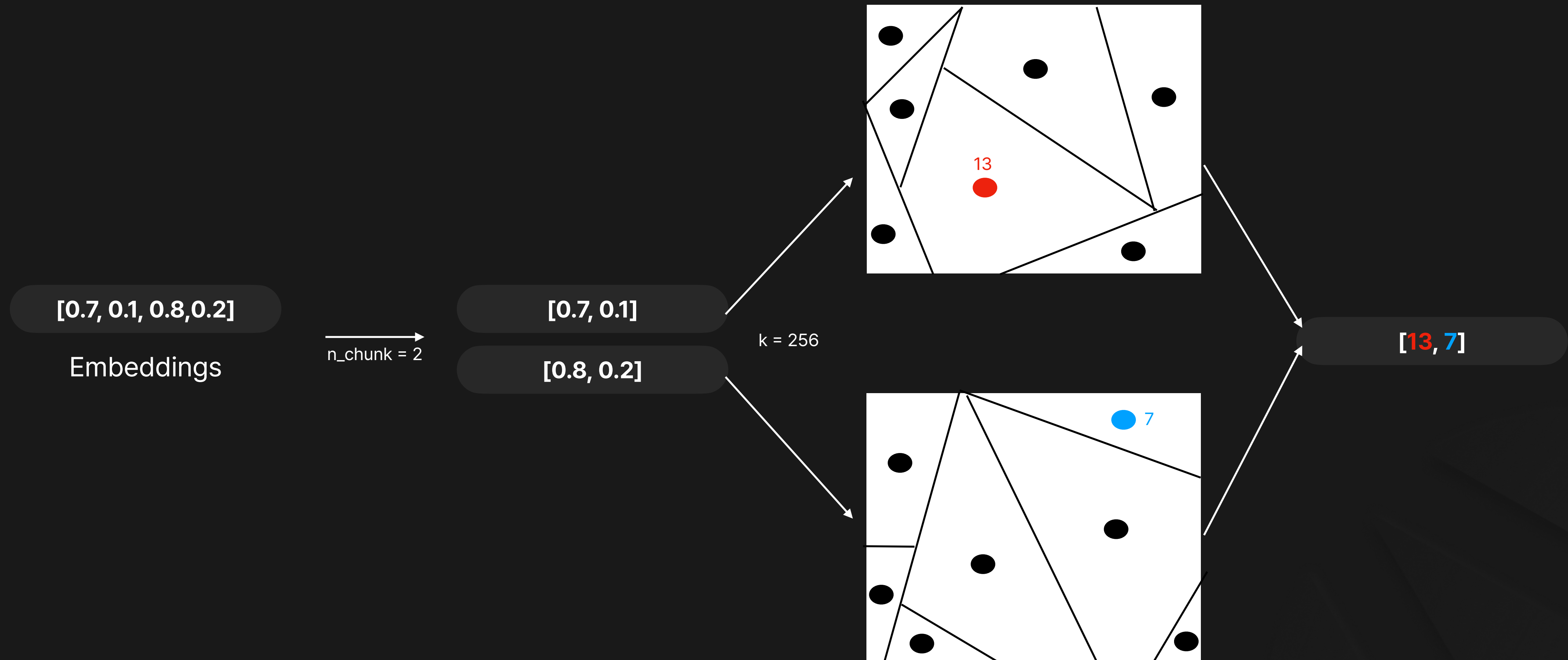
Scalar Quantization(SQ)

- 데이터 컴프레션 테크닉 중 하나로, float 형식으로 구성된 데이터 포인트를 Integer로 변환시키는 방식.
- 단순히 소수점을 정수로 Rounding하는 것에서 그치는 것은 아님
- 우리가 인덱싱 하려는 벡터들의 값 분포를 토대로 값이 자주 등장하는 구간을 정의하고(보통 95퍼센타일, 99퍼센타일 등), 해당 분포구간 내의 값들을 int8이나 uint8 범위로 매핑시켜 변환하는 방식
- 인터벌 바깥의 Outlier 값들은 각각의 인터벌 민맥스 값으로 클리핑 시키게됨
- 메모리 최적화 용도이기 때문에 4바이트 float을 1바이트 integer로 변환 시 속도 대폭 개선 가능



Product Quantization(PQ)

- Scalar Quantization 기법과 마찬가지로 임베딩 벡터의 floating 포인트들을 8비트 integer로 변환시켜 메모리 효율성을 증진시키는 기법
- 여기에 몇가지 추가적인 기술이 가미된 기법이 PQ
- 원본 벡터를 n개의 청크, 혹은 서브벡터 단위로 분할시키게 됨.
- 분할된 서브벡터 단위로 각각 K-Means 클러스터링을 진행 (K = 256)
- 클러스터링 결과로 나온 256개의 Centroid들을 서브벡터별의 representation으로 매핑시켜 양자화 완료
- 신규 쿼리 벡터가 들어오면 해당 벡터를 사전에 정의한 서브벡터 개수로 나누고, 각 서브벡터별 매핑되는 센트로이드 형태로 변환하게 되면 8비트 integer의 형태가 되고, 이후 거리계산 수행 시 메모리 효율성과 검색 정확도 둘다 챙기는 방식
- 덜 쪼갤수록 압축율은 올라가지만 정확도가 내려가는 트레이드오프 관계



Product Quantization(PQ)

Method	Class name	index_factory	Main parameters	Bytes/vector	Exhaustive
Exact Search for L2	IndexFlatL2	"Flat"	d	4*d	yes
Exact Search for Inner Product	IndexFlatIP	"Flat"	d	4*d	yes
Hierarchical Navigable Small World graph exploration	IndexHNSWFlat	"HNSW,Flat"	d, M	$4*d + x * M * 2 * 4$	no
Inverted file with exact post-verification	IndexIVFFlat	"IVFx,Flat"	quantizer, d, nlists, metric	4*d + 8	no
Locality-Sensitive Hashing (binary flat index)	IndexLSH	-	d, nbits	ceil(nbits/8)	yes
Scalar quantizer (SQ) in flat mode	IndexScalarQuantizer	"SQ8"	d	d	yes
Product quantizer (PQ) in flat mode	IndexPQ	"PQx", "PQ"M"x"nbits	d, M, nbits	ceil(M * nbits / 8)	yes
IVF and scalar quantizer	IndexIVFScalarQuantizer	"IVFx,SQ4" "IVFx,SQ8"	quantizer, d, nlists, qtype	SQfp16: $2 * d + 8$, SQ8: $d + 8$ or SQ4: $d/2 + 8$	no

Product Quantization(PQ)

Querying

Keyword Search

- 전통적 방식의 키워드 기반 검색.
- 문서의 의미적 유사성을 구하는 것이 아닌, 단어의 등장 빈도가 비슷한 문서를 검색하거나, 등장 여부를 필터링하여 검색하는 방식
- Attribute Filter, Sparse vector search로 나누어짐

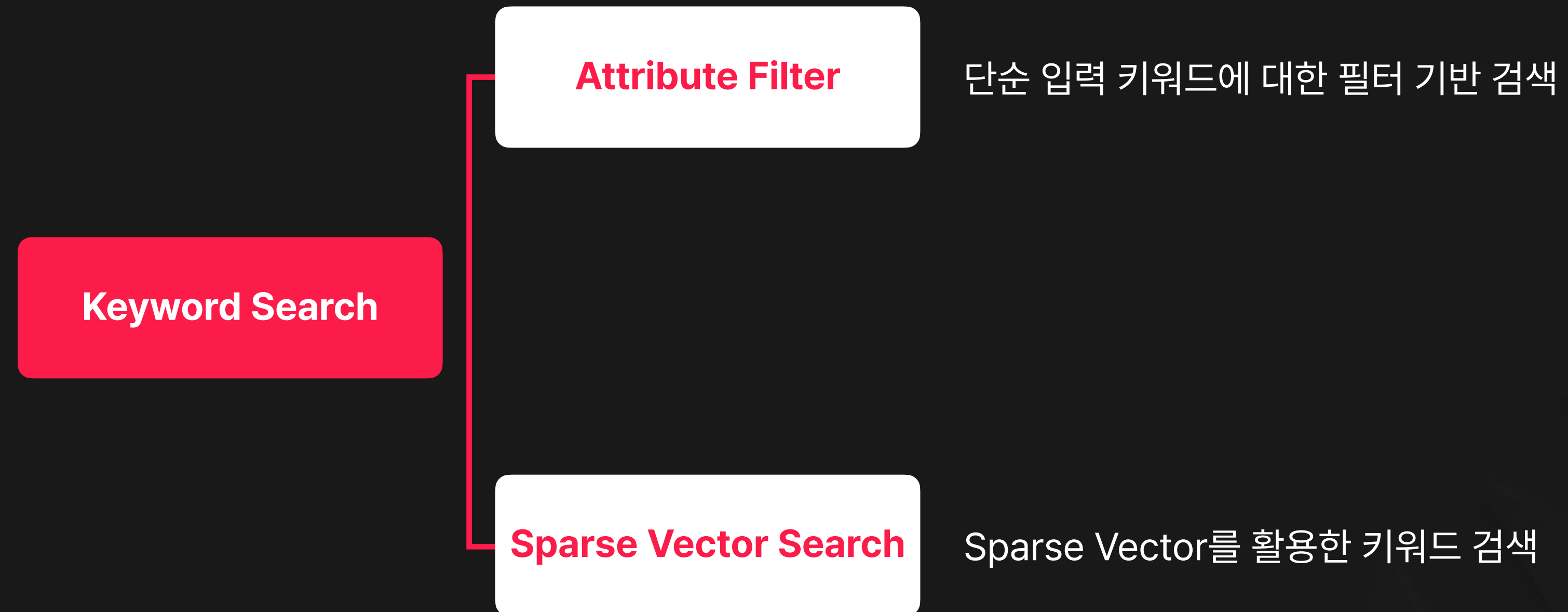
Semantic Search

- 벡터 데이터베이스의 기본 검색방식.
Dense 하게 표현된 벡터들을 cosine 유사도, 유클리드 거리, dot-product 등의 거리계산 식을 이용하여 의미적으로 유사한 문서를 검색하는 방식

Hybrid Search

- Keyword search와 semantic search를 조합하여 활용하는 방식
- 상호간의 단점을 보완하는 형태로使用时 가장 우수한 성능 보장
- 굉장히 창의적이고 다양한 방법들로 구현 가능

Querying

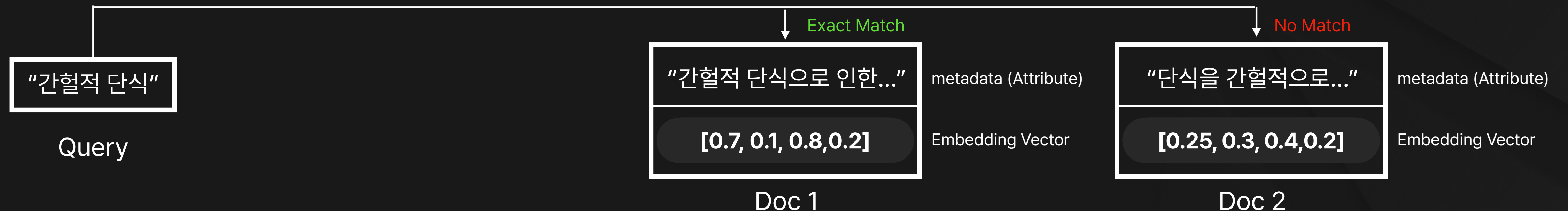


Keyword Search - Attribute Filtering

- 입력 키워드에 등장하는 단어가 DB 내의 데이터들에서 exact match 방식으로 단순 등장여부를 기준으로 검색
- 벡터 데이터베이스가 아닌 컨텍스트에서는 일반적으로 키워드 필터 기반 검색이라고도 함
- 벡터 데이터베이스 컨텍스트에서는 해당 키워드 필터를 수행하기 위해 메인 데이터인 임베딩 벡터가 아닌, 해당 벡터를 기준으로 하는 부수적인 정보로써 인덱싱되고, 그걸 기반으로 필터링 하기 때문에 Attribute Filtering이라고 함.

장점: 직관적이다. 빠르다. 검색 요건의 자율성이 제한될수록 정확해진다. **고유명사가 서칭 키워드일 시 훨씬 정확하다.**

단점: 일반적으로 부정확하다. 유저의 검색실력에 결과품질이 좌우된다.



Keyword Search - Sparse Vector Search

- 단순 키워드 필터 기반의 검색에서 한단계 더 나아간 키워드 기반 검색 방식
- 키워드를 단순히 등장하면 결과로 주고, 등장하지 않으면 결과로 안주는 키워드 필터와는 다르게, 대상 문서들에 대해 단어 은행을 만들고, 해당 단어 은행에 기반한 n-gram 형태로 문서들을 벡터로 나타냄
- sparse하다고 표현하는 이유는 실제 문서를 이 방식으로 표현 시 1 이상의 숫자값을 가진 차원보다 0값이 훨씬 많기 때문.
- 단어은행 상의 단어들이 스파스 벡터 차원별로 매핑되어 등장 빈도가 표현되기 때문에 의미적 유사성까진 아니더라도 기본적인 수준으로 문서의 토픽을 모델링하는 효과를 가져가는 것이 가능
- 대표적인 스파스 벡터 서칭 모델로는 BM25와 SPLADE 가 있음

Query **"Are people from east less likely to speak up?"**

	↓	↓	↓	↓	↓	↓	↓	↓	↓
BoW Index	3	31	96	82	2	78	33	67	53

Doc 1 [0, 7, 0, 0, 2, 3, 1, 0, 0]

Doc 2 [10, 3, 5, 3, 0, 3, 2, 4, 0]

Top Match

Doc 3 [0, 0, 0, 0, 2, 0, 0, 0, 0]

Keyword Search - Sparse Vector Search : BM25

- TF-IDF 메커니즘 활용하는데, TF(Term Frequency)는 위 설명한 n-gram기반 단어등장빈도에 대해 문서레벨로 표현
 - IDF(Inverse Document Frequency)는 해당 문서의 단어가 다른 문서들에서 얼마나 등장하지 않는 단어인지에 따라 가중치를 부여하며 조정
→ 사용자가 물어보는 단어가 가지고 있는 문서 내에서도 언급하는 문서가 몇개 없을 시, 그 중 그럼에도 불구하고 해당 단어를 가장 많이 언급하는 문서가 그만큼 높은 확률로 사용자의 질문과 연관성이 높을 것이라는 가정
 - 이후 사용자의 입력 쿼리를 모든 문서들에 대조시켜 BM25 스코어를 계산하고, 높은 스코어를 가진 문서를 결과로 출력
- Dense Vector를 이용하지 않기 때문에 벡터 데이터베이스가 아닌 컨텍스트에서 사용 가능하며, 벡터 데이터베이스에서 활용 시 sparse vector를 따로 인덱싱시켜서 활용 필요

장점: 필터기반 대비 단어 언급 수에 기반하므로 더욱 연관성 있는 문서 발췌 가능, **고유명사가 서칭 키워드일 시 훨씬 정확하다.**

단점: 필터기반과 마찬가지로 유연성 떨어짐(오타에 취약), 이에 따른 사용자 검색 실력에 따라 결과 성능 달라짐

Query **"Are people from east less likely to speak up?"**



BoW Index 3 31 96 82 2 78 33 67 53

Doc 1 [0, 7, 0, 0, 2, 3, 1, 0, 0]

Doc 2 [10, 3, 5, 3, 0, 3, 2, 4, 0]

Doc 3 [0, 0, 0, 0, 2, 0, 0, 0, 0]

Top Match

Keyword Search - Sparse Vector Search : SPLADE

- Sparse Vector Search의 단점인 유연성을 개선하기 위해 Sparse Vector 생성 시 BERT모델을 활용하는 방식.
- 중요도에 따른 가중치 부여: 단어의 단순 등장 빈도가 아닌, 해당 단어가 문서에서 차지하는 중요도를 스코어로 표현하여 스파스하게 표현.
- Term Expansion : BERT모델과 결합한 구조로 단어나 단어의 조합들이 의미적으로 유사한 표현들로 포괄적으로 확장
 - 오타, 간접적 표현 등도 term expansion을 통해 캐치 가능해짐에 따라 어이없는 mismatch 상황 방지 가능
 - 단순 빈도가 아니라 중요도 가중치로 문서 표현함으로 쿼리벡터와 문서간 토큰단위 중요도가 동일하게 높은 문서를 찾는 것이 가능

단점: 다른 sparse vector search 방식 대비 현저히 느림

Query	SPLADE BOW rep	New terms (expansion)	Dropped terms (compression)
Most Dangerous Vehicles	(dangerous, 2.33), (most, 2.21), (vehicle, 2.09), (car, 1.85), (worst, 1.19), (truck, 1.01), (danger, 0.52), (top, 0.38)	top, danger, worst, truck, car, vehicle	vehicles

Semantic Search

- 벡터 데이터베이스에서 다루는 메인 검색 기능.
- 문서의 의미적 유사성을 거리계산 함수를 이용하여 벡터간 거리를 계산하여 결과 출력을 하는 방식
- 검색하고자 하는 기반 인덱스에서 구축 시 사용한 인덱싱 알고리즘에 따라 거리 계산이 수행되는 지점이 달라짐.
(Ex. LSH 기반 인덱스 → 해싱함수 통과 이후 유클리디안 거리계산 사용)
(Ex. HNSW+PQ 기반 인덱스 → PQ 선 진행 이후 graph traverse 시 거리계산 함수 반복 사용)
- 자주 사용되는 거리계산 식은 총 세가지가 있는데 각각 Euclidean Distance, Cosine similarity, Dot product Similarity이다.
- 인덱스 내 인덱싱된 두 벡터를 어떤 거리 계산식을 이용하는지에 따라 그 거리가 다르게 나올 수 있음
- 어떤 거리계산 식을 사용할 것인지에 대한 결정은 항상 해당 임베딩 벡터의 벡터화를 진행한 임베딩 모델이 학습 시 사용한 계산식을 그대로 사용하는 것이 Best.

Distance Metrics

"Semantic Distance 구하기"

Euclidean Distance

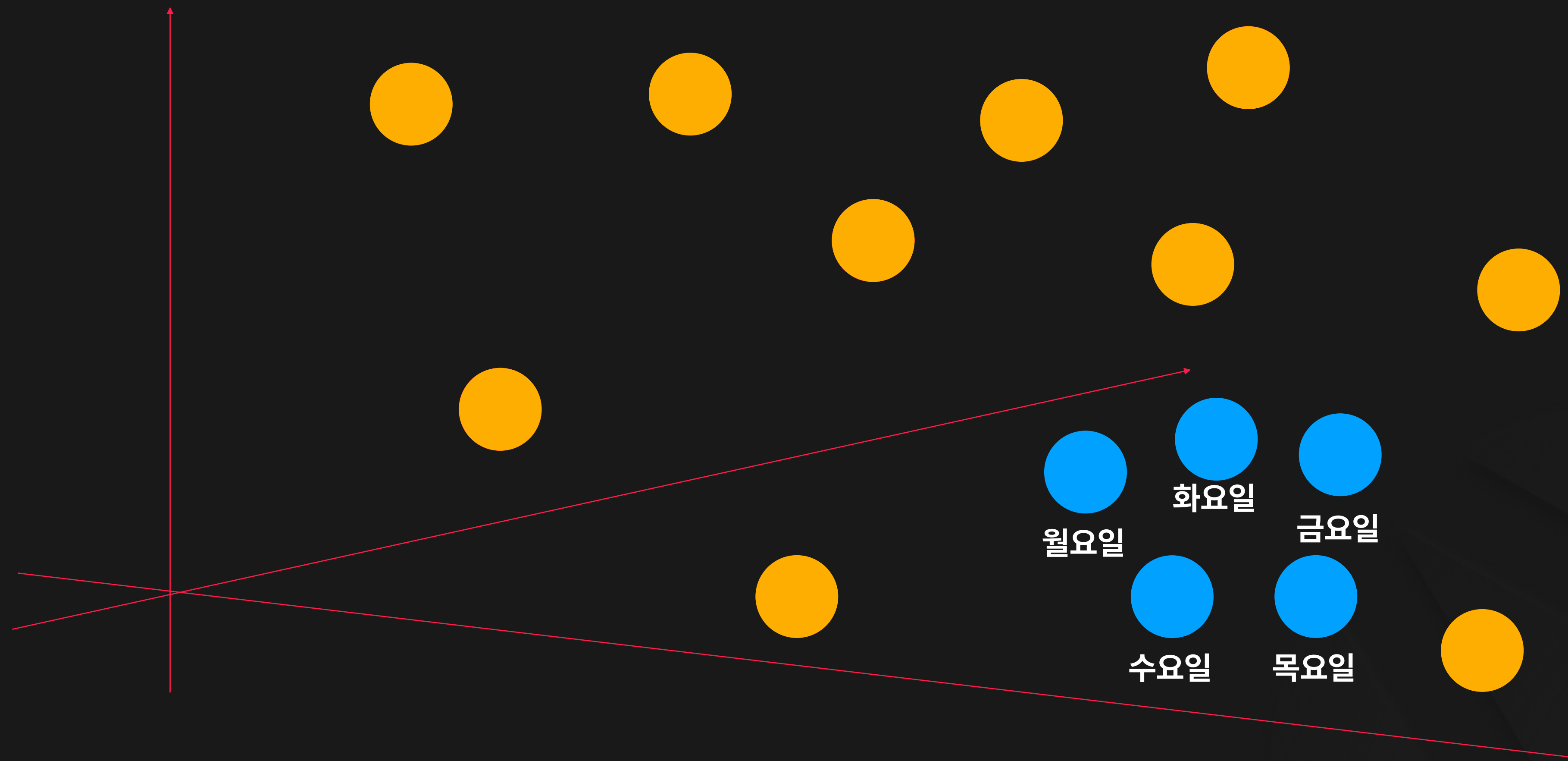
- 벡터의 차원별 대응값들의 거리를 구해서 합산하여 표현하는 방식
- 스케일에 민감하며 방향성과 거리감을 둘다 표현가능
- 단, 벡터 간 길이가 다를 시 사용하면 안됨
- 딥러닝 기반 임베딩에는 잘 안쓰이며, 빈도수 같은 스파스 벡터 형태에 활용하기 적합

Dot Product

- 두 벡터간의 내적.
- 벡터간 각도가 90도 이하일 시 양수, 90도 이상일 시 음수, 90도일 시 0
- 방향성 거리감 둘다 표현 가능
- 다수 LLM모델들이 내적으로 학습시켰기 때문에 내적 사용을 추천

Cosine Similarity

- 내적 결과에서 magnitude를 나눈 2차적 결과
- magnitude를 상쇄시키기 때문에 거리감은 알려주지 않지만 방향성을 알려줌
- -1 부터 1의 스케일로 표현되며, 1에 가까울수록 유사한 방향성, -1에 가까울수록 반대
- 픽셀 인텐시티와 같은 강도 관련 요소가 임베딩 될 시 활용하면 안됨



“문맥상 비슷한 위치에 등장하는 단어는 비슷한 의미를 가진다”

Keyword Search (Attribute Filter / Sparse Vector)

vs

Semantic Search (Dense Vector)

장점

- 속도가 빠르다
- 비용 효율적이다
- 제한적 검색 요건을 가질 때 정확하다
- 의미보다 단어의 생김새가 중요할 때 정확하다(고유명사)

단점

- 유연성이 떨어진다
- 간접적 표현에 대한 인지력이 떨어진다
- 검색 쿼리의 디테일에 검색 성능이 의존적이다

- 정확한 표현이 아니어도 잘 알아먹을 수 있다
- 오타로부터 자유롭다
- 유사함에 기반한 결과 소팅이 가능하다
- 멀티모달을 지원한다(이미지, 텍스트, 오디오 등)

- 속도가 느리다
- 리소스 헤비하다
- 고유명사 등 제한적 컨텍스트에서나 많이 쓰는 단어에 대해 약하다

Hybrid Search

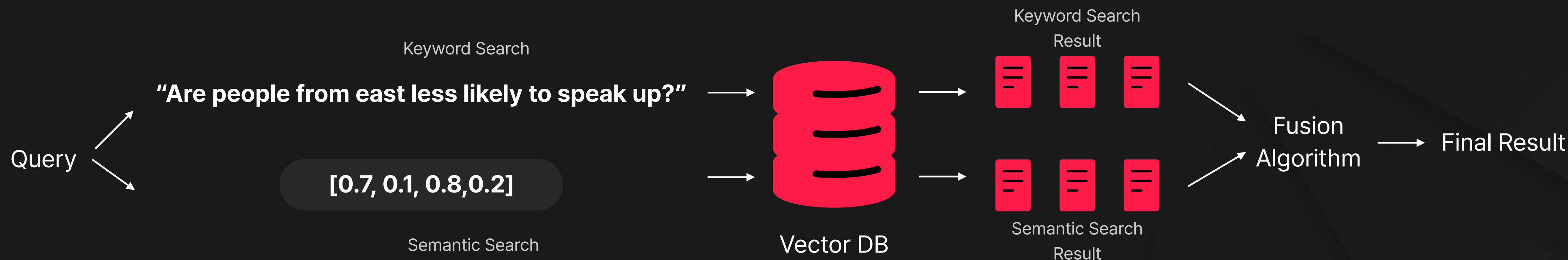
- 벡터 데이터베이스로부터 정보 검색을 할 때 키워드 서칭 방식과 Semantic 서칭 방식을 조합하여 상호 간의 장점만 취하는 방향으로 더욱 안정적인 성능을 노리는 방식. (이외 멀티모달 컨텍스트에서는 텍스트 + 이미지 조합으로 하는 서칭도 해당)

- 굉장히 자유로운 방식의 서칭 조합 사용 가능

→ "OO일보에서만 한 달 전 발간한 금리 관련 내용을 찾아줘" → Attribute Filter(발간지, 발행일) + Semantic Search

"축구 관련 기사를 찾는데, 그 중에서도 해외파 선수들의 소속팀에서의 성적과 관련된 부분을 찾아줘" → SPLADE(해외파 선수들 → 손흥민, 김민재 등) + Semantic Search

- intermediate 결과를 토대로 하나의 서칭결과로 합치는 퓨전 알고리즘 필요



Hybrid Search

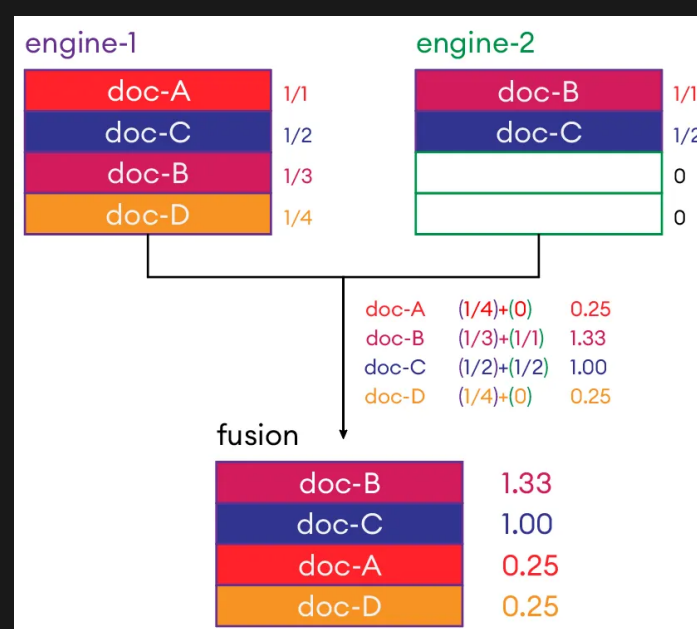
- Sparse + Dense 서칭 결과를 토대로 최종 결과를 생성하기 위해 Fusion 스코어링 필요

Naive weighted score : $0.3 * \text{sparse 스코어} + 0.7 * \text{Dense 스코어}$

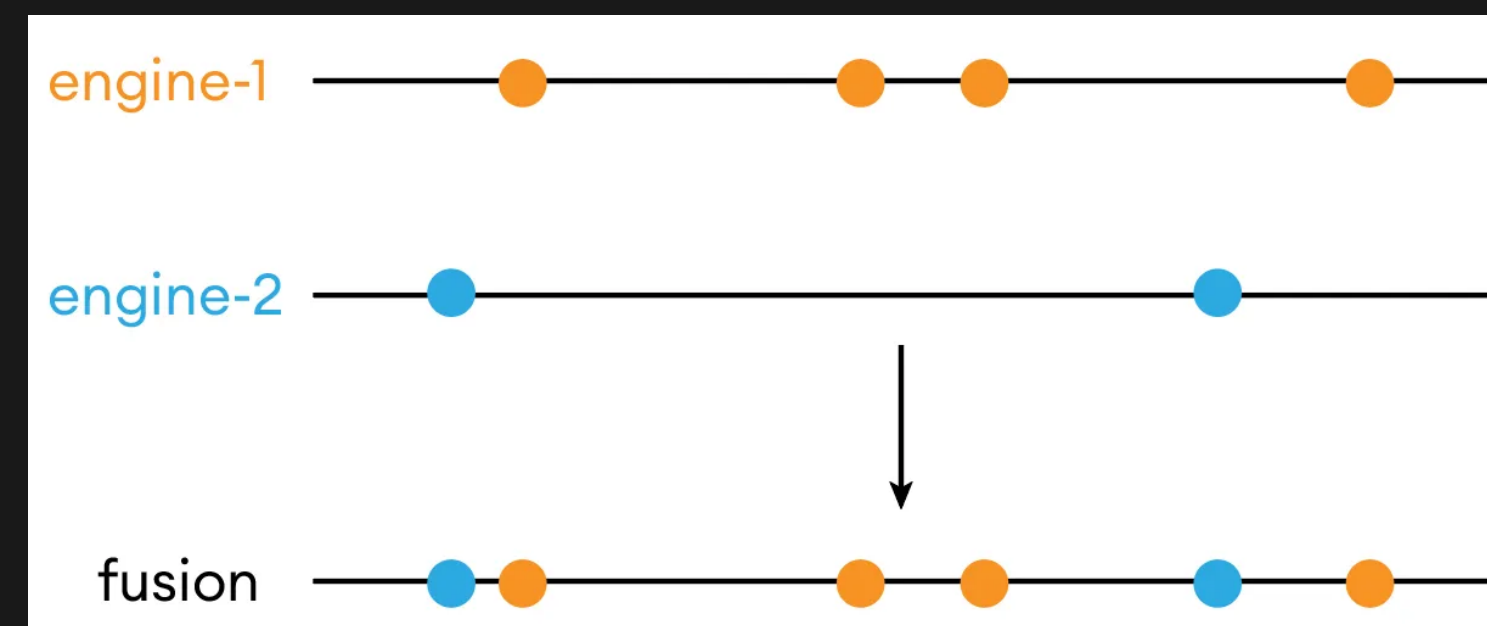
Reciprocal Ranked Fusion(RRF) : 두 결과리스트에서의 순위적 위치만을 가지고 결과 스코어로 퓨전하는 방식

Relative Score Fusion(RSF) : 두 결과를 normalize시킨 후 줄세워서 결과를 통합시키는 방식

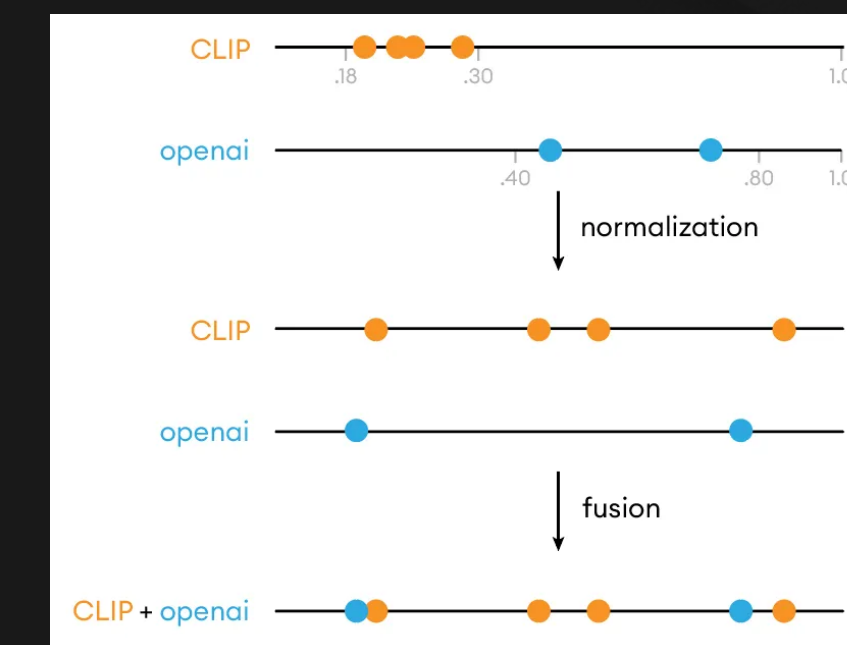
Distribution-based score Fusion(DBSF) : 두 결과를 뽑은 엔진의 분포가 서로 상이해도 유의미한 score fusion이 가능하도록 분포 기반 스케일링을 추가한 방식



RRF



RSF



DBSF

Vector Database 종류

Vector Databases



LanceDB



drant



Weaviate



Pinecone



vespa



Milvus



"뭘 써야되는거지?"

벡터 데이터베이스 선택 시 고려 사항

- 작성 언어
 - : 어떤 언어로 작성이 된 데이터베이스인지에 따라 속도 / 확장성 측면에서 차이 발생(Golang, Rust, Java, C/C++)
- 오픈 소스 여부
 - : 오픈 소스 기반 데이터베이스가 일반적으로 선호됨
- 호스팅 옵션
 - : 셀프 호스팅, 매니지드 클라우드 서비스 등 지원 범위에 따라 현실적 제약을 고려했을때의 선택지가 달라짐
- 인덱싱 알고리즘
 - : 솔루션별로 임베딩 벡터 인덱싱 시 사용 가능한 인덱싱 알고리즘의 종류
- 다큐먼트 수준
 - : 참고용
- 커뮤니티 활성화도
 - : 사례 참조, 막혔을때 도움을 구할 수 있을지, 기능 추가/수정 등

Pinecone

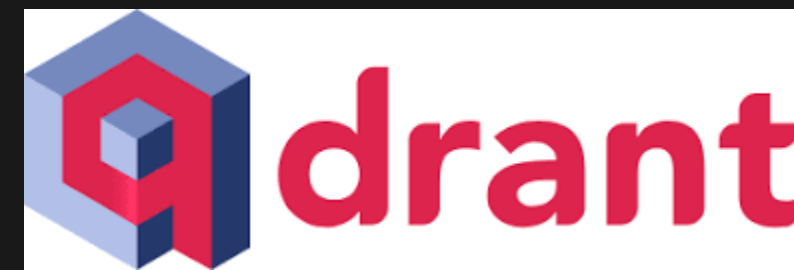


<https://community.pinecone.io/>

- 작성 언어 : Rust
- 오픈소스 여부 : X
- 호스팅 옵션 : Cloud Only(Managed)
- 인덱싱 알고리즘 : Brute Force Search 시(KNN) FAISS 활용, 이외 ANN index 메서드의 경우 공개되지 않음
- 다큐먼트 수준: 괜찮은 편
- 커뮤니티 활성화도 : 낮은 편

- 장점: Fully-managed 서비스로 바로 사용 가능. 하이레벨 api로 초보자도 개념적 이해 없이 사용하기 쉽게 구성.
- 단점: 오픈소스가 아니다 보니 가려진 부분이 많음. Managed service에 따른 DB셋업, 운영, 관리에 대한 컨트롤 가능 영역 적음.

Qdrant



<https://github.com/qdrant/qdrant>

- 작성 언어 : Rust
- 오픈소스 여부 : O
- 호스팅 옵션 : Cloud or Self-host
- 인덱싱 알고리즘 : Custom HNSW
- 다큐먼트 수준: 매우 깔끔
- 커뮤니티 활성화도 : 매우 높음 (Github Star 16.7k)

- 장점: Rust, Python, Golang 등 가장 많이 사용되는 백엔드 언어 API 제공, 참고 가능 예시 굉장히 많음
- 단점: 상대적 후발주자로 Reference가 적은 편

Vald



<https://github.com/vdaas/vald>

- 작성 언어 : Golang
 - 오픈소스 여부 : O
 - 호스팅 옵션 : Cloud or Self-host
 - 인덱싱 알고리즘 : NGT(Neighborhood Graph and Tree)
 - 다큐먼트 수준: 불친절한 편
 - 커뮤니티 활성화도 : 낮음 (Github Star 1.4k)
-
- 장점: 매우 빠른 ANN 알고리즘 채용(NGT), 멀티모달 데이터 처리에 적합한 분산처리 구조
 - 단점: 활성화도가 낮고 사례 찾기가 어려움

Weaviate



<https://github.com/weaviate/weaviate>

- 작성 언어 : Golang
- 오픈소스 여부 : O
- 호스팅 옵션 : Cloud or Self-host
- 인덱싱 알고리즘 : HNSW (PQ)
- 다큐먼트 수준: 깔끔한 편
- 커뮤니티 활성화도 : 높음 (Github Star 9.1k)

- 장점: 매우 큰 데이터셋 기반의 확장성 강조(엔터프라이즈 포커스), 참조 가능 레퍼런스 다수
- 단점: 확장성 가져가기 위해 K8s 사용 및 이에 따른 Extra 인프라적 관리 포인트

Vespa



<https://github.com/vespa-engine/vespa>

- 작성 언어 : Java + C/C++
- 오픈소스 여부 : O
- 호스팅 옵션 : Cloud or Self-host
- 인덱싱 알고리즘 : HNSW, BM25
- 다큐먼트 수준: 괜찮은 편
- 커뮤니티 활성화도 : 중간 (Github Star 5.2k)

- 장점: 벡터DB 선조격으로 HNSW+BM25 하이브리드 서치 기능을 통한 다수 레퍼런스 보유
- 단점: Rust, Golang로 작성된 후발주자 대비 아쉬운 속도

Milvus



<https://github.com/milvus-io/milvus>

- 작성 언어 : Golang
- 오픈소스 여부 : O
- 호스팅 옵션 : Self-host or cloud(via zilliz)
- 인덱싱 알고리즘 : IVF, HNSW, DiskANN 등 가장 많이 지원
- 다큐먼트 수준: 나쁘지 않음
- 커뮤니티 활성화도 : 최상 (Github Star 25.9k)

- 장점: 온디스크 인덱스용 SOTA인 DiskANN 지원, 이외 가장 많은 인덱싱 알고리즘 선택 가능
- 단점: 파이썬 유저로써는 상대적으로 사용하기 불편한 API

LanceDB



<https://github.com/lancedb/lancedb>

- 작성 언어 : Rust
- 오픈소스 여부 : O
- 호스팅 옵션 : cloud
- 인덱싱 알고리즘 : IVF(PQ), DiskANN
- 다큐먼트 수준: 좋음
- 커뮤니티 활성화도 : 발전 중 (Github Star 2.4k)

- 장점: DiskANN지원, Rust로 작성 및 랜스 데이터 포맷 활용한 속도 최적화
- 단점: 가장 최근에 나와 레퍼런스 가장 적음

Pinecone을 활용한 Indexing & Searching 실습

Pinecone



<https://pinecone.io>

- 작성 언어 : Rust
- 오픈소스 여부 : X
- 호스팅 옵션 : Cloud Only(Managed)
- 인덱싱 알고리즘 : Brute Force Search 시(KNN) FAISS 활용, 이외 ANN index 메서드의 경우 공개되지 않음
- 다큐먼트 수준: 괜찮은 편
- 커뮤니티 활성화도 : 낮은 편

- **장점: Fully-managed 서비스로 바로 사용 가능. 하イレ벨 api로 초보자도 개념적 이해 없이 사용하기 쉽게 구성.**
- **단점: 오픈소스가 아니다 보니 가려진 부분이 많음. Managed service에 따른 DB셋업, 운영, 관리에 대한 컨트롤 가능 영역 적음.**

Pinecone

Pinecone Record

Record ID

'vector-1'

Dense vector

.1, .2, .3, .4...

Sparse vector

Indices

Values [.5, .5, .2...]

Metadata

Key1 : Value1

Key2 : Value 2

- Record ID
: ID기반 operation 용도(수정, 제거, 조회 등)
- Dense Vector
: Semantic Search 용도
- Sparse Vector
: Keyword Search / Hybrid Search 용도
- Metadata
: Attribute filter / Hybrid Search 용도

